INFO 4000 Mini Project 2 — Part 1

Deep Q-Learning for Inventory Management

Business Scenario

Imagine that you are part of a small manufacturing company's operations analytics team. The company purchases raw materials, converts them into finished products, and sells those products to customers. Managers must decide how to use available cash and inventory without knowing exactly what future market conditions will be.

Buying materials too early ties up resources and creates inventory holding costs. Buying too late can leave the company unable to fulfill an order. Manufacturing a product requires sufficient raw materials, and customer demand may arise before a finished product is available. Meanwhile, raw-material prices and product selling prices fluctuate.

At the start of each operating period, a production planner must decide whether to spend cash on another material unit, turn available material into a sellable product, or hold steady. Customer orders are small and sporadic, and every decision uses up the opportunity to take a different action in that period. The company wants to explore whether an **AI decision-making agent** can learn an effective sequence of these decisions through repeated interaction with a simulated business. Rather than prescribing a fixed rule such as “always buy” or “always manufacture,” the agent should learn when to **wait, purchase, or produce** based on the current inventory, prices, demand, and cash.

You have been given a simplified simulation of this operation, implemented as a custom **Gymnasium environment**. Although it is intentionally smaller than a real manufacturing system, it captures the essential challenge of *sequential decision-making*: an action taken now can affect which actions are possible—and how much reward is earned—later.

Objective

Your assignment is to implement and train a **Deep Q-Network (DQN)** agent that learns a policy for managing the inventory system. Your goal is to achieve strong cumulative reward by balancing sales opportunities, material purchases, inventory holding costs, and penalties for unmet demand or invalid decisions.

You will then evaluate the trained agent in new simulation episodes to determine whether it makes sensible operational decisions and how its performance affects cash. The focus is on **building and testing the RL process**, not on redesigning the environment or building a production-ready manufacturing system.

1. Files Provided

- **`Inventory_env_class.py`** — contains `InventoryManagementEnv`, `NormalizeObservation`, `ReplayBuffer`, and `DQN`.
- **`requirements.txt`** — lists the Python packages required to run the exercise.

Treat the supplied environment as a Gymnasium environment, just like the environments used previously in class. **Do not modify `Inventory_env_class.py`.** Write your implementation in your own Jupyter notebook.

2. Environment Reference (Gymnasium-style)

Environment ID/class: `InventoryManagementEnv` (a custom Gymnasium environment; instantiate the class directly).
Purpose: Learn when to wait, buy raw materials, or manufacture finished products while balancing sales opportunities, purchasing costs, inventory holding costs, and unmet-demand penalties.
Episode length: By default, up to **500 steps**.
Starting conditions: **$1,000 cash**, zero raw materials, and zero finished products.
Observation space: `Box` with shape `(6,)`, numeric `float32` features.
Action space: `Discrete(3)` (actions 0, 1, and 2).

Observations

The environment returns six features in this order:

Position	Feature	Description
0	`raw_inventory`	Raw-material units available
1	`product_inventory`	Finished-product units available
2	`raw_price`	Current cost to buy one raw-material unit
3	`product_price`	Current selling price per finished product
4	`demand`	**Current-period** customer demand, either 0 or 1 product
5	`cash`	Cash available to fund purchases

The observation is a continuous-valued array even though inventory and demand are integers. The provided `NormalizeObservation` wrapper scales the values for training; it does not change the underlying business rules.

Actions

The agent selects **one action per step**:

Action	Name	Effect
`0`	Wait	No purchase or conversion
`1`	Buy	Purchase **one** raw-material unit, if cash is sufficient
`2`	Convert	Use **two** raw-material units to make **one** finished product, if available

Products are **sold automatically** if customer demand and finished-product inventory are both present; there is no separate sell action. An unavailable purchase or conversion incurs an invalid-action penalty.

How a step works

1. The agent observes the current inventories, prices, demand, and cash, then selects an action.
2. The environment applies the purchase, conversion, or wait decision.
3. It automatically sells available finished products against **the demand in the observed state**.
4. It computes the reward, updates cash and inventory, and generates next-period prices and demand.
5. It returns `(next_observation, reward, terminated, truncated, info)`.

A finished product takes at least **two purchase actions and one conversion action** to create from zero raw inventory, so the agent must learn to plan across steps.

Market dynamics

Demand is intermittent: in the default configuration, the probability of a one-unit order fluctuates around **0.25** per step. Raw-material and product prices also fluctuate. These are intentionally simplified stochastic assumptions, not a forecast of any real manufacturer's market.

Reward

The step reward combines:

Component	Reward effect
Revenue from automatic product sales	Positive
Raw-material purchase costs	Negative
Valid conversion bonus	Small positive reward (default `+1`)
Holding cost for remaining raw and finished inventory	Negative
Unmet customer demand	Stockout penalty (negative)
Invalid purchase or conversion attempt	Penalty (negative)

Reward is not the same as cash or accounting profit. Cash changes when the agent buys raw material or sells products. Stockout penalties and holding costs reduce the reward but are *not charged against cash*. This distinction is important when evaluating results.

Episode endings and diagnostic information

An episode normally ends after the maximum number of steps (`truncated=True`). The environment also checks for cash depletion (`terminated=True`), but under the default rules, purchases require enough cash and bankruptcy is not normally reachable. When training DQN, stop an episode on either flag, but suppress target-value bootstrapping only for **true termination**.

The `info` dictionary provides useful diagnostics such as `sold_units`, `unmet_demand`, `revenue`, `purchase_cost`, `holding_cost`, and `invalid_action`. The `render()` method prints current inventory, prices, next-period demand, and cash; use it sparingly during evaluation. **After `step()`, the displayed demand is for the next period**, not the demand just served.

3. Getting Started

Import the supplied components and create a normalized environment:

```
from Inventory_env_class import (
    InventoryManagementEnv,
    NormalizeObservation,
    ReplayBuffer,
    DQN,
)

env = NormalizeObservation(InventoryManagementEnv())
obs, info = env.reset(seed=42)
```

Create a **policy network** and a **target network**, using the supplied `DQN` class. The networks should accept six input features and produce three Q-values, one for each action. Use the supplied `ReplayBuffer` class to store and sample experiences.

You may inspect the simulation through:

```
env.render()
```

Do **not** render every step during training, since it produces excessive output. Use rendering when examining the learned policy during testing.

4. Implementing the DQN Agent

Build the training process in your notebook. Include the following components:

1. **Epsilon-greedy action selection:** select between exploration and the current network's preferred action, using a schedule that reduces epsilon as training progresses.
2. **Experience replay:** store transitions and sample mini-batches for neural-network training.
3. **Bellman targets and loss:** estimate target Q-values using the target network, and optimize the policy network using mean squared error (MSE).
4. **Target-network updates:** periodically synchronize target-network parameters from the policy network.
5. **Episode handling:** stop an episode on either `terminated` or `truncated`. When masking future Q-values in the Bellman target, use **true termination**, rather than treating the time limit alone as a terminal state.

Choose and document your hyperparameters, including the number of training episodes, discount factor (`gamma`), learning rate, batch size, epsilon schedule, and target-network update frequency.

5. Training and Monitoring

Start with a brief run to confirm that the code executes correctly, and then carry out a longer training run. The required number of episodes is **not fixed**; explain the setting you use. Full DQN runs can take substantial time.

During training, record and periodically display:

- Total cumulative reward **per episode**.
- Epsilon at that stage of training.
- Average MSE loss over the training updates made during that episode.

Provide plots showing the **episode reward** and **average episode MSE** against the correctly labeled episode number. A rolling mean of reward is useful for identifying the general trend.

Interpreting results: Loss may fluctuate because DQN targets change during learning. A decreasing or stable MSE is **not proof** of a good policy. Use evaluation performance and observed decisions to judge whether the agent has learned useful behavior.

6. Evaluating the Trained Policy

After training, evaluate the learned policy over **10 complete, independent episodes**, each starting with **$1,000**. Disable epsilon exploration for these evaluation runs so the DQN acts greedily.

For **each** evaluation episode, report:

- Total cumulative reward.
- Final cash balance.
- Number of finished products sold.
- Total unmet demand.
- Number of invalid actions.

Use `env.render()` to display several consecutive steps from at least one test episode (for example, the first eight steps of episode 1). You do not need to render every step of every episode.

Summarize your results with the **mean reward**, **mean final cash**, and the **number of episodes finishing above the initial $1,000**. A comparison with a random-action policy is encouraged because it provides a useful baseline.

Demand and prices are random, so a good policy need not increase cash in **every** episode. Explain what your results show, including variability between episodes.

7. What Does Success Look Like?

A successful project demonstrates a **working DQN learning process and a defensible evaluation**, not a particular reward number.

1. **Correct implementation:** The training loop uses exploration, replay sampling, a policy network, Bellman targets, and target-network updates correctly.
2. **Evidence of learning:** The rolling average episode reward generally improves from early training. Individual episodes can vary substantially because market conditions are random.
3. **Sensible decisions:** The trained agent makes a coherent mix of buying, converting, and waiting decisions and avoids repeated invalid actions. Explain its observed choices using the state and business rules.
4. **Business outcomes:** Report cumulative reward **and** final cash separately over 10 test episodes. Count how many finish above the initial $1,000, and examine sales, unmet demand, and invalid actions.

5. **Comparison with a baseline:** Ideally, the trained policy achieves higher **mean episode reward** than a random-action policy on comparable test episodes. A random-policy comparison is strongly encouraged, but it is not an additional mandatory grading item.
6. **Reasoned interpretation:** Explain what the plots and evaluation support, including any limitations or episodes with weak performance.

Important: A decreasing MSE alone does **not** prove successful learning. Nor does success require positive reward or cash growth in every episode. There is **no fixed minimum score** to reproduce; the goal is to demonstrate learning and interpret the results appropriately. The instructor has observed improvement over a random baseline in one reference run, but stochastic outcomes will differ across students.

8. What to Submit

Submit **one well-organized Jupyter notebook** containing:

1. Environment and network setup.
2. DQN training implementation, with meaningful comments.
3. Training progress output and plots.
4. Ten-episode policy evaluation, including a rendering example and summary metrics.
5. Answers to the reflection questions below.

Due date: November 5th, 2026 midnight.

9. Reflection Questions

1. **Training experience:** What were the biggest challenges in training your DQN agent? What were your most important takeaways?
2. **Policy performance:** How well did the trained policy manage inventory over the 10 evaluation episodes? Discuss cumulative reward and final cash **separately**, and compare with a random policy if you implemented that baseline.
3. **Science and engineering applications:** Identify one or two situations in your field where reinforcement learning might be useful. Explain what the agent would observe, what actions it could take, and what outcome it should try to improve.

10. Grading Criteria

Criterion	Weight
Environment is set up correctly and the notebook executes successfully	**30%**
DQN training loop is implemented correctly as specified	**30%**
Ten-episode policy testing, reported results, and `render()` demonstration	**20%**
Meaningful reflection answers supported by reasoning	**15%**
Clear notebook structure and useful explanatory comments	**5%**
Total	**100%**

This assignment uses an intentionally simplified inventory simulator. Its purpose is to teach sequential decisions, delayed consequences, and evaluation of a learned DQN policy—not to reproduce all of the operational and financial constraints of a real manufacturing business.

MP2 Part 2: Digital Twin Assignment: PINN Control of a VFD Centrifugal Pump (With Data Ingestion)

System Overview The objective of this assignment is to model a flow-control loop featuring a centrifugal pump driven by a variable-frequency drive (VFD). You will control this asset using a Physics-Informed Neural Network (PINN) acting as an inverse-dynamics solver to calculate the required motor RPM for a target head setpoint. To accurately reflect industrial telemetry, you will build a simulated data ingestion module that injects realistic Gaussian noise into the sensor readings. This will demonstrate the critical interplay between noisy AI control commands and physical hardware constraints (VFD ramp rate limiting).

Development Requirements Build a Python-based Digital Twin utilizing PyTorch for the AI controller and core Python for the physical simulation. Implement the following components:

- **Data Generation Pipeline:** Sample the pump's controllable speed range (800-1800 rpm) and apply the algebraic system equations to generate 5,000 synthetic state-aware samples. The inputs are Target Setpoint, Current Speed, and Current Head; the label is the required RPM command.
- **PINN Architecture:** Design a 3-input Multi-Layer Perceptron (MLP) with two 64-node hidden layers and a single continuous output for the speed command.
- **Physics-Informed Training Loop:** Train the network using a combined loss function. Include a standard data loss and a physics loss that pushes the predicted RPM through the differentiable fluid dynamics equations. Penalize the network heavily if the resulting physical head fails to match the target setpoint.
- **Data Ingestion Module:** Build a `SimulatedPressureSensor` class that receives the mathematically ideal head from the physics engine and injects Gaussian noise (e.g., standard deviation of 0.25) before passing it to the PINN controller.
- **Digital Twin Integration & Simulation:** Run a 60-second dynamic simulation (0.1s time step) featuring a setpoint step change from 25 m to 15 m at t = 1 second.
    - The PINN must calculate its speed command based on the noisy ingested sensor data.
    - Implement a VFD adapter that enforces a strict mechanical acceleration limit of 2000 rpm/s.
    - Calculate the exact analytical RPM required at each time step using the exact physics equations to serve as a validation baseline.
- **Please also go through the slides presented in class highlighting the details for building this project**

Deliverables Submit a Jupyter Notebook alongside a 2-page report detailing:

1. The architecture of your digital twin, including the data ingestion module and PINN design.
2. The code structure, focusing on the separation between physical plant, sensor ingestion, and AI control.
3. A 4-panel time-series plot displaying: Head Tracking (Setpoint vs. Noisy Measured), Speed Tracking (Actual, Noisy PINN Command, and Analytical Command), Flow Rate, and Power/Specific Energy.

4. An analysis of the system's dynamic response, specifically explaining how the VFD's physical ramp limit acts as a low-pass filter to protect the physical motor from the high-frequency noise generated by the stateless PINN controller.

Grading Criteria

1. The application is developed as specified and fufills all the requirments and runs erro free - 50%
2. The Classes developed for the Digital twin are functional and well documented within the Class code - 10%
3. The simulation is well developed and does exactly what is expected of it - 20%
4. The report is intutive to understand and elaborates the main points - 10%
5. The questions are answered and the code is well commented - 10%

Due Date

November 5th, 2026 by midnight.